Experiment No. 6

Error Correction at Data Link Layer

Aim

To implement the Hamming Code technique for error detection and correction at the Data Link Layer and test it using different data streams.

Procedure

1. Start Python or another suitable programming environment and write a program to implement Hamming Code.
2. Enter the binary data stream that needs to be transmitted.
3. Calculate the number of parity bits required using the formula:

(2^r \geq m+r+1)

where (m) is the number of data bits and (r) is the number of parity bits.

4. Place the parity bits at positions that are powers of two, such as 1, 2, 4, and 8. Place the data bits in the remaining positions.
5. Calculate the parity bits according to the selected even-parity or odd-parity method. Insert the calculated parity bits into the codeword.
6. Display the generated Hamming code and consider it as the transmitted data.
7. To test error detection, introduce an error by changing one bit of the transmitted codeword.
8. At the receiver, recalculate the parity checks and determine the syndrome value. The syndrome identifies the position of a single-bit error.
9. If the syndrome is non-zero, correct the bit at the indicated position. If the syndrome is zero, no error is detected by the parity checks.
10. Display the received codeword, syndrome value, corrected codeword, and recovered data bits. Repeat the process with different data streams and error positions.

Input.txt:

HELLO WORLD

Sender.c:

```
#include
<stdio.h>
#include
<stdlib.h>
#include
<string.h>

int calculateParityBits(int dataBits)
{
  int r = 0;

  while ((1 << r) < (dataBits + r
    + 1)) r++;

  return r;
}

void encodeHamming(unsigned char data, FILE *channel)
{
  int hamming[13] = {0};
  int parityPositions[] = {1, 2, 4, 8};

  int dataIndex = 0;
```

```
// Store data bits in positions other than
powers of 2 for (int pos = 1; pos <= 12;
pos++)
{
  if (pos == 1 || pos == 2 || pos == 4 || pos
    == 8) continue;

  hamming[pos] = (data >> (7 - dataIndex)) & 1;
```

```
    dataIndex++;
  }

  // Calculate parity bits
  // P1 checks positions 1,3,5,7,9,11
  hamming[1] = hamming[3] ^ hamming[5] ^
          hamming[7] ^ hamming[9] ^
          hamming[11];

  // P2 checks positions 2,3,6,7,10,11
  hamming[2] = hamming[3] ^ hamming[6] ^
          hamming[7] ^ hamming[10] ^
          hamming[11];

  // P4 checks positions 4,5,6,7,12
  hamming[4] = hamming[5] ^ hamming[6] ^
          hamming[7] ^ hamming[12];

  // P8 checks positions 8,9,10,11,12
  hamming[8] = hamming[9] ^ hamming[10] ^
          hamming[11] ^ hamming[12];

  // Write 12-bit Hamming code to
  channel for (int i = 1; i <= 12; i++)
    fprintf(channel, "%d", hamming[i]);
}

int main()
```

```
{
  FILE
  *input;
  FILE
  *channel;


  char ch;
```

```
  input = fopen("input.txt", "r");

  if (input == NULL)
  {
     printf("Error: Could not open
     input.txt\n"); return 1;
  }

  channel = fopen("channel.txt", "w");

  if (channel == NULL)
  {
     printf("Error: Could not create
     channel.txt\n"); fclose(input);
     return 1;
  }

  printf("========== HAMMING CODE SENDER ==========\n\n");

  printf("Input text:\n");

  while ((ch = fgetc(input)) != EOF)
  {
 putchar(ch);
  }

rewind(input
```

```
                );

while ((ch = fgetc(input)) != EOF)
{
```

```
    encodeHamming((unsigned char)ch, channel);
  }

  fclose(input);
  fclose(channel
  );

  printf("\n\nHamming encoded data has been written to channel.txt\n");

  printf("\nChannel data:\n");

  channel = fopen("channel.txt", "r");

  while ((ch = fgetc(channel))
    != EOF) putchar(ch);

  fclose(channel);

  printf("\n\nSender completed successfully.\n");

  return 0;
}
```

Receiver.c:

```
#include
<stdio.h>
#include
<stdlib.h>
#include
<string.h>

int main()
{
  FILE *channel;
  char
  bits[100000];
  int length = 0;

  channel = fopen("channel.txt", "r");

  if (channel == NULL)
  {
    printf("Error: Could not open
    channel.txt\n"); return 1;
  }

  // Read channel
  data char ch;
```

```
while ((ch = fgetc(channel)) != EOF)
{
   if (ch == '0' || ch == '1')
   {
      bits[length++] = ch;
   }
}
```

```
bits[length] = '\0';

fclose(channel);

printf("========== HAMMING CODE RECEIVER ==========\n\n");

printf("Received channel data:\n%s\n\n", bits);

if (length % 12 != 0)
{
   printf("Error: Invalid Hamming
   data.\n"); return 1;
}

printf("Checking for errors...\n\n");

printf("Corrected/Decoded output:\n");

for (int block = 0; block < length; block += 12)
{
   int hamming[13];

   // Convert characters into
   integer bits for (int i = 1; i <=
   12; i++)
   {
```

```
   hamming[i] = bits[block + i - 1] - '0';
}

// Calculate syndrome
int p1 = hamming[1] ^ hamming[3] ^ hamming[5] ^
```

```
    hamming[7] ^ hamming[9] ^ hamming[11];

    int p2 = hamming[2] ^ hamming[3] ^
         hamming[6] ^ hamming[7] ^
         hamming[10] ^ hamming[11];

    int p4 = hamming[4] ^ hamming[5] ^
         hamming[6] ^ hamming[7] ^
         hamming[12];

    int p8 = hamming[8] ^ hamming[9] ^
         hamming[10] ^ hamming[11] ^
         hamming[12];

    int errorPosition = p1 * 1 +
              p2 * 2
              + p4 *
              4 + p8
              * 8;

    if (errorPosition == 0)
    {
printf("[No error in block %d] ", (block / 12) + 1);
    }
    else if (errorPosition <= 12)
    {
       printf("[Error in block %d at position %d -
```

```
        Corrected] ", (block / 12) + 1,
        errorPosition);

    // Correct the error
    hamming[errorPosit
    ion] =
      hamming[errorPosition] ^ 1;
}
else
```

```
{
        printf("[Invalid error detected] ");
    }

    // Extract data
    bits int data = 0;

    int
      dataPositions[]
      = { 3, 5, 6, 7,
      9, 10, 11, 12
    };

    for (int i = 0; i < 8; i++)
    {
      data = (data << 1) | hamming[dataPositions[i]];
    }

    printf("%c", (char)data);
  }

  printf("\n\nReceiver completed successfully.\n");

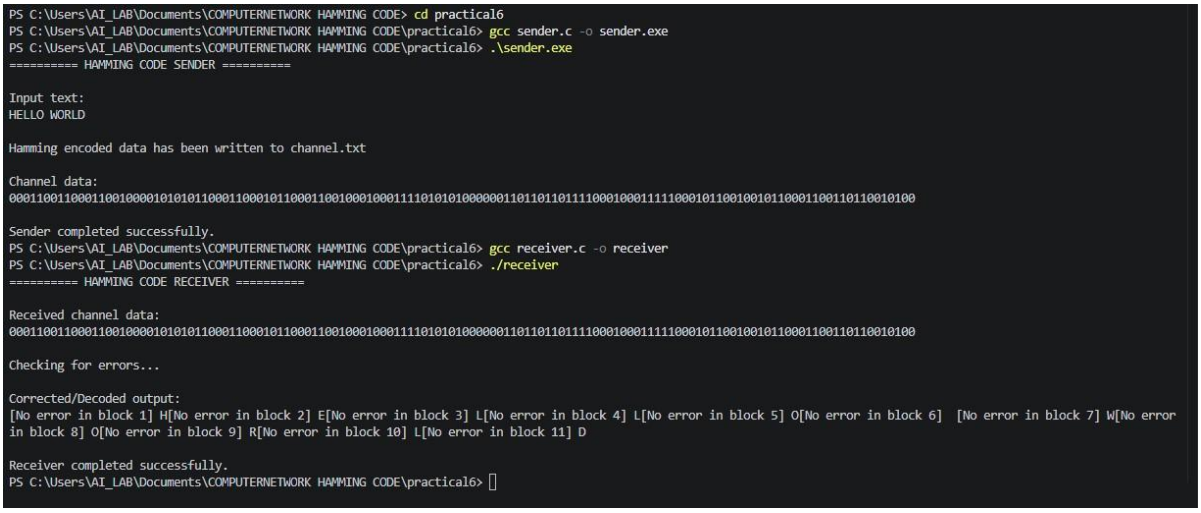
  return 0;
}
```

For compilation and

running: gcc sender.c -o

sender

./sender



Result

The Hamming Code technique was implemented successfully. Parity bits were generated and used to detect and correct single-bit errors in the received data stream, demonstrating error control at the Data Link Layer.